UTN FRLP

Desarrollo de Software 2026

Relaciones y Validaciones

Clase #17 — 24 de agosto de 2026

Matías Corti - Martina García Améndola



Dónde quedamos

La clase pasada dejamos la API con datos persistidos. La deuda que declaramos, y qué pasa con ella hoy:

Pendiente de C16	**Lo resuelve**
Libro.autor es String, sin relación	← HOY · @relation
POST sin título guarda basura	← HOY · Zod
/libros/abc da 404 en vez de 400	← HOY · validación
Diez try/catch, y todo error es 500	← HOY · middleware
Dos autores iguales dan 500, no 409	← HOY · errores de Prisma
Cualquiera borra cualquier libro	C18 · Auth con JWT
Tu front sigue leyendo libros.json	C19 · front ↔ back

Temas de hoy

- **Relaciones**: por qué autor como texto no alcanza
- **Los tres tipos**: 1:1, 1:N, N:M — cómo reconocerlos
- **include**: pedir el autor junto con el libro
- **Validar del lado del server**: Zod otra vez, el mismo de C10
- **Middleware**: por qué el orden importa, y cómo se van los try/catch
- **Práctica**: migrar datos existentes, devolver los status correctos



El problema: autor es un texto

Así quedó el modelo en C16:

```
model Libro {
  titulo String
  autor  String   // ← "Gabriel García Márquez", escrito a mano en cada libro
}
```

Quiero…	**Por qué no puedo**
Los libros de García Márquez	Un typo (Marquez, García Márquez) y el libro desaparece de la lista
Corregir el nombre de un autor	Está copiado en 4 libros: hay que tocar 4 filas y no olvidarse ninguna
Mostrar la nacionalidad del autor en el detalle del libro	El libro no sabe qué autor es. Sólo tiene su nombre escrito



Clave primaria, clave foránea, integridad

Tres conceptos que sostienen todo lo demás:

La diferencia con el texto: `autor: "García Márquez"` es una **copia**. `autorId: 3` es una **referencia**. La copia se puede escribir mal; la referencia, no: si el 3 no existe, la base rechaza la operación.

Concepto	Qué es	En nuestro modelo
Clave primaria (PK)	La columna que identifica una fila de forma única. Una por tabla	Libro.id, Autor.id
Clave foránea (FK)	Una columna que guarda la PK de otra tabla. Es el puntero	Libro.autorId → Autor.id
Integridad referencial	La base garantiza que la FK apunte a algo que existe	No podés guardar un libro con autorId: 9999

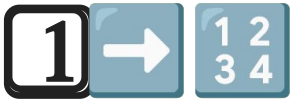


Los tres tipos de relación

Cómo reconocerlo: preguntá en las dos direcciones. "¿Un autor cuántos libros?" → muchos. "¿Un libro cuántos autores?" → uno. **uno ↔ muchos = 1:N**

Si las dos son "muchos", es N:M. Si las dos son "uno", es 1:1.

Tipo	Se lee	Ejemplo	Dónde va la FK
1:1	uno tiene exactamente uno	Usuario ↔ Perfil	En cualquiera de los dos, con @unique
1:N	uno tiene muchos	Autor → Libros	Siempre en el lado N (en Libro)
N:M	muchos con muchos	Libros ↔ Categorías	En una tabla intermedia



1:N

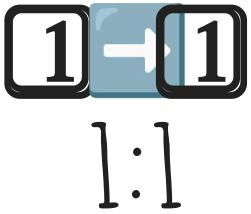
LA RELACIÓN DE HOY

- `autorId Int` es la **columna real** en `Libro`.
- `autor Autor @relation(...)`
- `libros Libro[]` NO son columnas: son el atajo de Prisma para navegar la relación
- `fields` = la columna de *esta* tabla, `references` = la columna de *la otra*

La FK va del lado N. Siempre. Un libro tiene un autor: el puntero lo lleva el libro.

```prisma
model Autor {
  id           Int     @id @default(autoincrement())
  nombre       String  @unique
  nacionalidad String
  libros       Libro[]          // ← lado "muchos": NO crea columna
}

model Libro {
  titulo   String
  autorId  Int                  // ← la FK: esto SÍ es una columna
  autor    Autor   @relation(fields: [autorId], references: [id])
}
```



1:1 es 1:N con @unique y el objeto singular (no Array).

Un usuario tiene un perfil, y ese perfil es de un solo usuario.

```
model Usuario {
  id        Int     @id @default(autoincrement())
  email     String  @unique
  perfil    Perfil?            // ← singular y OPCIONAL, no un array
}

model Perfil {
  id        Int    @id @default(autoincrement())
  bio       String
  usuarioId Int    @unique  // ← la FK. @unique es lo que la vuelve 1:1
  usuario   Usuario   @relation(fields: [usuarioId], references: [id])
}
```



N:M implícita

Libro ↔ Categoría

Un libro puede estar en varias categorías, y una categoría tiene varios libros. Las dos direcciones dicen "muchos".

```
model Libro {
  categorias Categoria[]
}

model Categoria {
  id     Int     @id @default(autoincrement())
  nombre String  @unique
  libros Libro[]
}
```

No hay FK en ninguna de las dos tablas: **Prisma crea una tercera tabla solo**, y la maneja él.

```
CategoriaToLibro
  A  →  Categoria.id
  B  →  Libro.id
  PRIMARY KEY (A, B)
```

Se llama **implícita** porque no la escribimos nosotros. La vas a ver en Prisma Studio y en el `migration.sql`, pero nunca en el `schema.prisma`.



N:M explícita

cuándo hace falta

La implícita sólo guarda **que** dos cosas están relacionadas. Si necesitás guardar **algo sobre la relación**, la tabla intermedia tiene que ser tuya:

```
model Alumno  {
  id Int @id @default(autoincrement())
  inscripciones Inscripcion[]
}

model Materia {
  id Int @id @default(autoincrement())
  inscripciones Inscripcion[]
}

model Inscripcion {    // ← la tabla intermedia, escrita por nosotros
  alumnoId  Int
  materiaId Int
  nota      Int                // ← el dato que vive EN la relación
  alumno    Alumno  @relation(fields: [alumnoId],  references: [id])
  materia   Materia @relation(fields: [materiaId], references: [id])
  @@id([alumnoId, materiaId])  // ← clave primaria compuesta
}
```

El criterio, en una pregunta: ¿hay algún dato que no es del A ni del B, sino *del vínculo entre los dos*? La nota no es del alumno ni de la materia: es de la inscripción. Fecha, cantidad, estado, precio al momento de la compra; todos son señales de N:M explícita.

🔄 Referential actions: qué pasa al borrar

Si borrás un autor que tiene libros, ¿qué pasa con los libros? Lo decidís vos, en el schema:

Acción	Qué hace	Cuándo
Restrict	No deja borrar si tiene libros	Default de Prisma en relaciones obligatorias
Cascade	Borra el autor y sus libros	El hijo no tiene sentido sin el padre
SetNull	Deja autorId = null en los libros	Sólo si la FK es opcional (Int?)

```
autor Autor @relation(fields: [autorId], references: [id], onDelete: Restrict)
```

No poner nada también es una decisión: si la relación es obligatoria, Prisma pone **Restrict**. Eso significa que el DELETE de un autor con libros va a fallar. Bien: es lo que queremos.

⚠️ **Cascade** es el más peligroso: un DELETE de una fila puede borrar cientos. Se usa a propósito, nunca por copiar y pegar.



INCLUDE: TRAER EL AUTOR CON EL LIBRO

```
// Sin include: sólo el libro
const libro = await prisma.libro.findUnique({ where: { id: 1 } });
// { id: 1, titulo: "Cien años...", autorId: 2 }

// Con include: el libro Y su autor, en UNA sola consulta
const libro = await prisma.libro.findUnique({
  where: { id: 1 },
  include: { autor: true },
});
// { id: 1, titulo: "Cien años...", autorId: 2,
//   autor: { id: 2, nombre: "Gabriel García Márquez", nacionalidad:
"Colombia" }}
```

Ojo con esto: cambió el contrato de la API.
El JSON que devuelve `GET /api/libros/1` ahora tiene un objeto anidado adentro. Cualquiera que consuma la API (su propio front) va a recibir una forma distinta a la de C16.

```
// Y funciona para los dos lados: el autor con todos sus libros
await prisma.autor.findUnique({
  where: { id: 2 },
  include: { libros: true }
});
```

`include` también acepta varias relaciones: `include: { autor: true, categorias: true }`.

⚠️

INCLUDE Y EL TIPO: EL BUG QUE COMPILA

Poner include en el service **compila igual** aunque el tipo no lo mencione.

```
// libro.service.ts - esto NO da error de compilación
export async function findAll(): Promise<Libro[]> {
  return prisma.libro.findMany({ include: { autor: true } });
}
```

El JSON de la respuesta **sí trae el autor**. Pero el tipo lo perdió, y explota cuando alguien lo quiere usar:

```
error TS2551: Property 'autor' does not exist on type
'{ id: number; titulo: string; ... autorId: number; }'.
Did you mean 'autorId'?
```

La solución es una línea:

```
import { Prisma } from "../generated/prisma/client";

export type LibroConAutor = Prisma.LibroGetPayload<{ include: { autor: true } }>;

export async function findAll(): Promise<LibroConAutor[]> {
  return prisma.libro.findMany({ include: { autor: true } });
}
```

LibroGetPayload<...> = "el tipo de un Libro tal como sale de *esta* consulta". Prisma lo genera solo: el schema sigue siendo la única fuente de verdad.

🔁

Mini-cierre: relaciones

Concepto	En una línea
PK / FK / integridad	PK identifica la fila; FK apunta a otra tabla; la base garantiza que exista
1:1	Uno con uno. FK con @unique. Poco frecuente
1:N	Uno con muchos. FK del lado N. onDelete: Restrict, Cascade o SetNull
N:M implícita	Prisma crea y maneja la tabla intermedia
N:M explícita	La tabla intermedia es nuestra: la relación tiene datos propios
include	Traer la relación en una sola consulta. Cambia el JSON
LibroGetPayload	El tipo de un Libro con su relación



¿QUÉ TIPO DE RELACIÓN ES?

De a uno. Votan a mano alzada: **1:1, 1:N o N:M**. El que vota dice por qué, preguntando en las dos direcciones.

5 minutos

1. Un **cliente** y sus **pedidos**
2. Un **alumno** y sus **materias**
3. Una **factura** y su **número de CAE** de AFIP
4. Un **pedido**, sus **productos**, y la **cantidad** de cada uno
5. Un **médico**, sus **pacientes**, y el **turno** en el que se ven
6. Un **libro** y su **editorial**



Actividad: modelando las relaciones del TP

En papel, por equipo. *15 minutos*

1. Escriban las **3 a 5 entidades principales** de su TP (las que ya tienen en el modelo de datos de C13)

2. Dibujen **flechas** entre las que estén relacionadas

3. Al lado de cada flecha, el **tipo**: 1:1, 1:N o N:M — validándolo con la pregunta en las dos direcciones

4. Para las N:M, decidan si es **implícita o explícita**: ¿hay algún dato del vínculo?

5. Traduzcan **una** de las relaciones a sintaxis Prisma, a mano

Listo cuando: cada flecha tiene su tipo y pueden justificarlo en voz alta, y tienen una relación escrita en sintaxis Prisma.

Al cierre: dos equipos leen una relación y la validamos entre todos.



Validación en el backend

El POST que su formulario nunca permitiría

El formulario no es la puerta, es una de las puertas. `curl`, Postman o las DevTools entran por al lado. Manden esto por el `api.http`:

```
POST {{baseUrl}}/api/libros
Content-Type: application/json

{ "precio": -500 }    // ← devolvió 201 Created
```

Lo que mandé	Lo que hizo C16
Sin título	Guardó el libro con el título vacío
precio: -500	Guardó un libro que te paga por comprarlo
precio: "gratis"	500, sin explicar nada
GET /libros/abc	404. Pero abc no es un id: es un 400



Los tres niveles de validación

Nivel	Para qué	Qué pasa si falta
1. Frontend	Feedback rápido, sin ir al server. No es seguridad	Error feo. Se saltea con DevTools
2. Backend	La que sí protege. El cliente no la evita	Entra cualquier cosa (filmina anterior)
3. Base de datos	`NOT NULL`, `@unique`, FKs. La última línea	Datos inconsistentes igual

Se llama defensa en profundidad. No es redundancia: cada nivel atrapa algo que los otros no.

Ejemplo con el mismo campo, `precio`:

- **Front** te avisa mientras escribís, sin recargar
- **Back** rechaza el `-500` que llegó por `curl`
- **DB** `NOT NULL` rechaza el `null` que se colara por un bug nuestro.



Validación ≠ regla de negocio ≠ constraint

Tres cosas distintas, que se resuelven en tres lugares distintos:

Qué es	Pregunta que responde	Dónde vive	Status si falla
Validación	¿El dato tiene la forma correcta?	Zod, en el middleware	400
Regla de negocio	¿Tiene sentido en mi dominio?	El service	404 / 409 / 422
Constraint	¿La base lo acepta?	El schema de Prisma	409 / 500

Con el mismo POST de un libro:

- `titulo` vacío → **forma** → Zod → 400
- `autorId: 9999` con formato válido pero el autor no existe → **negocio** → el service pregunta → 404
- Dos autores con el mismo `nombre` (`@unique`) → **constraint** → DB rechaza → 409.

Zod no puede validar que el autor exista: para saberlo hay que ir a la base. Y la base no puede validar que el título no esté vacío con un espacio: para eso está Zod.



Reutilización con Frontend

En front, dijimos: *"el mismo schema se reutiliza en el backend"*.

Lo que sí se comparte	**Lo que no**
La misma librería: zod@4.4.3, la misma versión	El archivo. Son dos proyectos, dos node_modules, dos package.json
La misma API: safeParse, error.issues, z.infer	El código: hay que escribirlo dos veces
El mismo criterio de qué es un dato válido	—



Zod, del otro lado

Es el mismo Zod de Front.

```
export const libroCreateSchema = z.object({
  titulo:  z.string().trim().min(1, "El título es obligatorio").max(200),
  precio:  z.number().int().positive("El precio debe ser mayor a 0"),
  imagen:  z.string().min(1, "La imagen es obligatoria"),
  autorId: z.number().int().positive("El autor es obligatorio"),
});
```

En C10 dijimos safeParse en el front y parse en el back. Lo corregimos: safeParse en los dos.

	parse	**safeParse**
Si falla	lanza excepción	devuelve { success: false, error }
En el back	El error se va por un camino que no controlamos	Decidimos nosotros qué status devolver

Con **safeParse** el middleware lee **resultado.error.issues** y arma un 400 prolijo. Con **parse** habría que envolver todo en **try/catch**.

⚠️ **error.issues, no error.errors.** errors era Zod 3.



Un schema por operación

POST y PUT no piden lo mismo. Dos schemas, uno derivado del otro:

```
// validations/libro.validation.ts
export const libroCreateSchema = z.object({ /* … los 4 campos … */ });

// Para actualizar: los mismos campos, todos opcionales
export const libroUpdateSchema = libroCreateSchema.partial();

// El tipo sale del schema, gratis
export type LibroCreate = z.infer<typeof libroCreateSchema>;
```

`.partial()` toma un schema y hace opcionales todos sus campos. Una línea, sin repetir reglas: si mañana el título pasa a `max(300)`, cambia en los dos.

⚠️ **La trampa: `.partial()` NO saca los `.default()`.**

```
z.object({ disponible: z.boolean().default(true) }).partial()
  .safeParse({ precio: 9000 })
// → { precio: 9000, disponible: true }   ← ¡apareció solo!
```

Un PUT que sólo cambia el precio te pisaría `disponible` en `true` sin que nadie lo pida. **Por eso los defaults van en el schema de Prisma (`@default(true)`), no en el de Zod.** En Zod, `disponible` va como `.optional()`.

🔢

Los params también son input

GET /api/libros/abc. En C16 devolvía **404**: Number("abc") → NaN → Prisma no encuentra nada. Pero abc no es un libro que no existe: **es un id inválido**. Eso es un 400.

z.coerce.number() convierte antes de validar: **todo lo que viene en la URL es string** (req.params.id es "3", no 3).

```
export const idParamSchema = z.object({
  id: z.coerce.number().int().positive("El id debe ser un número positivo"),
});
```

Request	Antes	Ahora
/api/libros/3	200	200
/api/libros/9999	404	404 ✔ (existe la ruta, no el libro)
/api/libros/abc	404 ✘	400 ✔
/api/libros/-5	404 ✘	400 ✔

🔁

Mini-cierre: validación

Concepto	En una línea
Los tres niveles	Front = UX · Back = seguridad · DB = integridad. Defensa en profundidad
Nunca confiar en el cliente	El front se saltea con DevTools o con curl
Validación / negocio / constraint	Zod → 400 · service → 404/409 · schema → 409
safeParse	Devuelve un valor, no una excepción. En los dos lados
error.issues	Zod 4. error.errors era Zod 3
.partial()	Deriva el update. Cuidado: conserva los .default()
z.coerce.number()	Todo lo que viene en la URL es string



Qué es un middleware

Ya usaron uno: `express.json()`. Un middleware es **una función que corre entre que llega el request y que responde tu handler.**

```
function miMiddleware(req: Request, res: Response, next: NextFunction)
{
  // hago algo con el request..
  next();   // ...y le paso el turno al siguiente
}
```

Express arma una **cadena** y la recorre en orden:

```
// request → express.json() → validate(schema) → controller → response
//                 |                  |
//                 └ parsea el body   └ valida el body
```

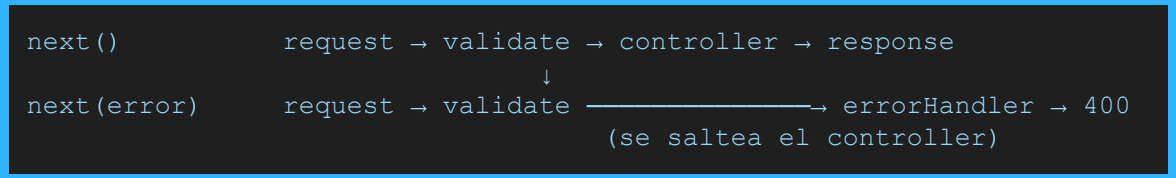
El orden es el orden en que los escribís. Por eso `express.json()` va antes de las rutas: si va después, cuando el controller lee `req.body` todavía no existe. Ese fue el bug de C15.

Cada middleware puede: leer el request, **modificarlo** (`express.json()` le agrega `body`), cortar la cadena respondiendo él mismo, o pasar el turno.

🔀 next() vs next(error)

Un middleware tiene dos formas de pasar el turno, y son muy distintas:

- `next()` = "todo bien, seguí con el próximo de la cadena".
- `next(error)` = "algo falló": SALTEA todo y va derecho al manejador de errores.



Si le pasás cualquier argumento a `next`, Express asume que es un error. No hay `next("ok")`.

Eso es lo que hace que el patrón cierre: el middleware de validación no necesita saber qué status devolver ni cómo se formatea el error. Sólo dice "esto falló" y sigue de largo. Quién traduce el error a HTTP es un solo lugar, y lo vemos ahora.



El middleware validate

```ts
// middlewares/validate.middleware.ts
import { Request, Response, NextFunction } from "express";
import { ZodType } from "zod";

export const validate = (schema: ZodType) =>
  (req: Request, _res: Response, next: NextFunction) => {
    const resultado = schema.safeParse(req.body);
                                  // ↓ 400, vía errorHandler
    if (!resultado.success) return next(resultado.error);
    req.body = resultado.data;    // ← el body ya validado
    next();
  };
```

- **Es una función que devuelve una función.** `validate(libroCreateSchema)` no valida: *fabrica* el middleware que valida con ese schema.
- **`req.body = resultado.data`** reemplaza el body con la versión validada y convertida. Del middleware para adelante, el controller puede confiar en lo que hay ahí.

```ts
// routes/libro.routes.ts
router.post("/",     validate(libroCreateSchema),
libroController.create);
router.put("/:id",   validate(libroUpdateSchema),
libroController.update);
```



errorHandler: un solo lugar para los errores

Un middleware de error tiene **cuatro parámetros**, y el primero es el error. **Express 5 le manda las promesas rechazadas solo:** los diez `try/catch` de C16 **se borran**, sin reemplazo.

```
// middlewares/error.middleware.ts
export const errorHandler = (
  err: unknown, _req: Request, res: Response, _next: NextFunction
) => {
  if (err instanceof ZodError) {
    return res.status(400).json({
      error: "Datos inválidos",
      detalles: err.issues.map(i => ({ campo: i.path.join("."), mensaje:
i.message }))
    });
  }
  console.error(err);
  return res.status(500).json({ error: "Error interno del servidor" });
};
```

```
// index.ts VA ÚLTIMO, después de todas las rutas
app.use("/api/libros", libroRoutes);
app.use("/api/autores", autorRoutes);
app.use(errorHandler);   // ← si va antes, no se ejecuta nunca
// El controller ya no necesita try/catch:
export async function create(req: Request, res: Response) {
  const nuevo = await libroService.create(req.body);
  return res.status(201).json(nuevo);
}
```

🔢

Los errores de Prisma, traducidos a HTTP

Cuando la base rechaza algo, Prisma lanza un error con código. El `errorHandler` los traduce:

```
if (err instanceof Prisma.PrismaClientKnownRequestError) {
  if (err.code === "P2002") return res.status(409).json({ error: "Ya existe un registro con ese valor" });
  if (err.code === "P2025") return res.status(404).json({ error: "No encontrado" });
  if (err.code === "P2003") return res.status(409).json({ error: "Hay registros relacionados" });
}
```

Código	Qué pasó	En la librería	Status
P2002	Violación de @unique	Dos autores, mismo nombre	409
P2025	El registro no existe	PUT /libros/9999	404
P2003	Violación de FK	autorId inexistente, o hijos	409/400

⚠️ **P2003 significa dos cosas distintas:** un `autorId` que no existe (400/404), o un `DELETE` con hijos por `Restrict` (409). El **service** debería preguntar antes; P2003 queda como red de seguridad.

🔁

Mini-cierre: middleware y status codes

La regla del 4xx vs 5xx: si el cliente puede arreglarlo cambiando el request, es 4xx. Si no, es 5xx y el problema es nuestro.

Código	Cuándo	Ejemplo
400	Dato mal formado	POST sin titulo · GET /libros/abc
404	El recurso no existe	GET /libros/9999
409	Choca con el estado actual	Autor duplicado, borrar con libros
422	Bien formado, imposible de procesar	Muchas APIs lo usan; nosotros usamos 400
500	Explotó nuestro código	Nunca por culpa del cliente



PRÁCTICA

💻

Paso 0: chequeo de arranque

5 minutos

```
# 1) Todo arriba
docker compose up -d
docker compose ps            # api y db arriba, db "healthy"

# 2) Zod adentro del container (lo instalaron la clase pasada)
docker compose exec api npm ls zod          # → zod@4.4.3

# 3) El cliente de Prisma, al día
docker compose exec api npx prisma generate

# 4) Los datos, ahí
docker compose exec db psql -U postgres -d libreria_db -c 'SELECT count(*)
FROM "Libro";'
```

⚠️ **Chequeen que el `.env` tenga estas dos líneas al final** (estaban en el `.env.example` de C16, pero el `.env` es de cada uno y no va a Git): `CHECKPOINT_DISABLE=1` y `PRISMA_HIDE_UPDATE_MESSAGE=1`.

Listo cuando: los dos containers arriba, `zod@4.4.3` responde, y el `SELECT count(*)` dice **10**.

🔴 **Si dice 0, corré `docker compose exec api npx prisma db seed` antes de seguir.** El paso 3 no tiene sentido sin datos.

Ejemplo Completo: DS26-Libreria (C17)



Paso 1: el seed, primero

10 minutos. Antes de tocar el schema.

```
// prisma/seed.ts
const autores = [{ nombre: "Antoine de Saint-Exupéry", nacionalidad:
"Francia" }, /* … los 10 */];
const categorias = [{ nombre: "Novela" }, { nombre: "Ensayo" }, { nombre:
"Técnico" }];
const libros = [
  { titulo: "El principito", autor: "Antoine de Saint-Exupéry",
    precio: 4500, imagen: "https://…", disponible: true, cats: ["Novela"] },
// ... los 10, SIN id
];
async function main() {
  await prisma.autor.createMany({ data: autores }); // 1) los autores primero
  await prisma.categoria.createMany({ data: categorias });
  for (const { autor, cats, ...datos } of libros) { // 2) después los libros
    await prisma.libro.create({ data: {
      ...datos,
      autor:      { connect: { nombre: autor } }, // ← por nombre: es @unique
      categorias: { connect: cats.map(nombre => ({ nombre })) },
    } });
  }
}
```

- **Autores primero.** Un libro necesita un `autorId` que exista, si no da `P2003`
- **`connect`** = usá el que ya existe
- **Sin `id` a mano.** El id lo pone la base

Ejemplo Completo: DS26-Libreria (C17)



Paso 2: el schema con las relaciones

10 minutos

```
model Autor {
  id           Int      @id @default(autoincrement())
  nombre       String   @unique
  nacionalidad String
  libros       Libro[]                    // ← NUEVO
}

model Libro {
  id          Int         @id @default(autoincrement())
  titulo      String
  precio      Int
  imagen      String
  disponible  Boolean     @default(true)
  autorId     Int                         // ← NUEVO: la FK
  autor       Autor       @relation(fields: [autorId], references: [id])   //
← NUEVO
  categorias Categoria[]                 // ← NUEVO ·  autor String ← SE VA
}

model Categoria {                        // ← NUEVO
  id     Int     @id @default(autoincrement())
  nombre String  @unique
  libros Libro[]
}
```

Y actualizar los tipos:

```
// services/libro.service.ts
export type { CategoriaModel as Categoria } from "../generated/prisma/models";
```

Ejemplo Completo: DS26-Libreria (C17)



Paso 3: la migración que Prisma se niega a hacer

12 minutos

```
docker compose exec api npx prisma migrate dev --name relaciones
```

Prisma corta en seco: le pedís agregar `autorId` obligatorio a una tabla que **ya tiene 10 filas**, y no hay autor posible para ellas. **En desarrollo, la salida es resetear.**

```
# 1) Borrar la base y re-aplicar las migraciones desde cero
docker compose exec api npx prisma migrate reset --force

# 2) Ahora sí: crear la migración nueva
docker compose exec api npx prisma migrate dev --name relaciones

# 3) El cliente TypeScript ← comando APARTE, no lo dispara migrate
docker compose exec api npx prisma generate

# 4) Los datos ← APARTE también: reset NO corre el seed
docker compose exec api npx prisma db seed

# 5) Verificar
docker compose exec db psql -U postgres -d libreria_db -c '\dt'
```

Ejemplo Completo: DS26-Libreria (C17)

⚠️

RESET ES DE DESARROLLO, NO DE PRODUCCIÓN

`migrate reset` **borra toda la base.**

En desarrollo no pasa nada; en producción sería el fin de la empresa.

Con datos reales, el mismo cambio se hace en tres migraciones incrementales:

#	Migración	Qué hace
1	autorId Int? nullable	Entra sin romper nada. Filas viejas quedan en null
2	Script de datos	Busca o crea el autor por nombre y completa el autorId
3	autorId Int obligatorio	Ya no hay null: entra sin conflicto



Paso 4: include en el service, y el tipo

10 minutos

```ts
// services/libro.service.ts
import { Prisma } from "../generated/prisma/client";

export type LibroConAutor = Prisma.LibroGetPayload<{ include: { autor: true } }>;

export async function findAll(disponible?: boolean): Promise<LibroConAutor[]>
{
  return prisma.libro.findMany({
    where: { disponible },
    include: { autor: true }
  });
}

// En el detalle, el autor Y las categorías
export type LibroDetalle = Prisma.LibroGetPayload<{
  include: { autor: true; categorias: true }
}>;

export async function findById(id: number): Promise<LibroDetalle | null> {
  return prisma.libro.findUnique({
    where: { id },
    include: { autor: true, categorias: true }
  });
}
```

⚠️ si TypeScript dice Property 'autor' does not exist... Did you mean 'autorId'?, falta el GetPayload en el tipo de retorno.

Ejemplo Completo: DS26-Libreria (C17)



Paso 5: los schemas de Zod

[Ejemplo Completo: DS26-Libreria (C17)]

10 minutos

Carpeta nueva: src/validations/. Y el equivalente para autores (nombre, nacionalidad).

```
// validations/libro.validation.ts

import { z } from "zod";

export const libroCreateSchema = z.object({
  titulo:     z.string().trim().min(1, "El título es obligatorio").max(200),
  precio:     z.number().int().positive("El precio debe ser mayor a 0"),
  imagen:     z.string().min(1, "La imagen es obligatoria"),
  disponible: z.boolean().optional(),          // el default lo pone Prisma
  autorId:    z.number().int().positive("El autor es obligatorio"),
});

export const libroUpdateSchema = libroCreateSchema.partial();

export const idParamSchema = z.object({
  id: z.coerce.number().int().positive("El id debe ser un número positivo"),
});

export type LibroCreate = z.infer<typeof libroCreateSchema>;
```

Y el equivalente para autores (nombre, nacionalidad).

Bonus: mensajes por defecto en español con z.config(z.locales.es()) en index.ts.

Paso 6: validate + el id de params

8 minutos

Carpeta nueva: src/middlewares/. Se pueden encadenar varios: el PUT valida primero el id, después el body.

```
// middlewares/validate.middleware.ts
export const validate = (schema: ZodType) =>
  (req: Request,  res: Response, next: NextFunction) => {
    const resultado = schema.safeParse(req.body);
    if (!resultado.success) return next(resultado.error);
    req.body = resultado.data;
    next();
  };
export const validateParams = (schema: ZodType) =>
  (req: Request, _res: Response, next: NextFunction) => {
    const resultado = schema.safeParse(req.params);
    if (!resultado.success) return next(resultado.error);
    next();
  };
```

```
// routes/libro.routes.ts
router.get(   "/:id", validateParams(idParamSchema),
libroController.getById);
router.post(  "/",    validate(libroCreateSchema),   libroController.create);
router.put(   "/:id", validateParams(idParamSchema),
validate(libroUpdateSchema), libroController.update);
router.delete("/:id", validateParams(idParamSchema), libroController.remove);
```

Se pueden encadenar varios: el PUT valida primero el id, después el body.

Ejemplo Completo: DS26-Libreria (C17)



Paso 7: el errorHandler y el borrado de try/catch

[Ejemplo Completo: DS26-Libreria (C17)]

10 minutos

```
// middlewares/error.middleware.ts
import { ZodError } from "zod";
import { Prisma } from "../generated/prisma/client" ;

export const errorHandler = (
 err: unknown, _req: Request, res: Response, _next: NextFunction
) => {
 if (err instanceof ZodError) {
   return res.status(400).json({
     error: "Datos inválidos",
     detalles: err.issues.map(i => ({ campo: i.path.join("."), mensaje: i.message
})),
   });
 }
 if (err instanceof Prisma.PrismaClientKnownRequestError ) {
   if (err.code === "P2002") return res.status(409).json({ error: "Ya existe un
registro con ese valor" });
   if (err.code === "P2025") return res.status(404).json({ error: "No encontrado" });
   if (err.code === "P2003") return res.status(409).json({ error: "Hay registros
relacionados" });
 }
 console.error(err);
 return res.status(500).json({ error: "Error interno del servidor" });
};
```

```
// index.ts agregar al final, DESPUÉS de las rutas:
app.use(errorHandler);
```

Y ahora borren. Los diez try/catch de libros y autores.

```
// El controller queda así, sin try/catch (antes: filmina 31):
export async function create(req: Request, res: Response) {
  const nuevo = await libroService.create(req.body);
  return res.status(201).json(nuevo);
}
```



Paso 8: api.http, los casos de error

5 minutos

El `api.http` crece: ahora provocamos los errores **a propósito.**

```
### 400 - sin título
POST {{baseUrl}}/api/libros
Content-Type: application/json

{ "precio": 5000, "imagen": "https://…", "autorId": 1 }

### 400 - id inválido
GET {{baseUrl}}/api/libros/abc

### 400 - el autor no existe (formato válido, dato imposible)
POST {{baseUrl}}/api/libros
Content-Type: application/json

{ "titulo": "Huérfano", "precio": 5000, "imagen": "https://…", "autorId":
9999 }

### 409 - autor duplicado
POST {{baseUrl}}/api/autores
{ "nombre": "Gabriel García Márquez", "nacionalidad": "Colombia" }

### 409 - borrar un autor con libros (onDelete: Restrict)
DELETE {{baseUrl}}/api/autores/1

### 404 - actualizar algo que no existe
PUT {{baseUrl}}/api/libros/9999
```

Ejemplo Completo: DS26-Libreria (C17)



¿Terminaste? Extensiones

1. **CRUD de categorías.** `/api/categorias` con sus cinco endpoints, validado. Es parte de la actividad individual: empezala acá

2. **`GET /api/autores/:id/libros`** — los libros de un autor. Ruta anidada, como vimos en C15

3. **Filtrar por categoría:** `GET /api/libros?categoria=Novela`. Pista: `where: { categorias: { some: { nombre } } }`

4. **Difícil:** hacé que `POST /api/libros` con un `autorId` inexistente devuelva **400 con el mensaje "El autor no existe"** en lugar del 409 genérico. ¿En qué capa va esa pregunta, y por qué no puede ir en Zod?

🔁

Mini-cierre: la práctica

Lo que hicimos	En una línea
El seed y el schema	Autores antes que libros, connect, sin id a mano, con @relation y Categoria[]
La migración imposible	Prisma se niega: reset + migrate + generate + seed, ninguno dispara al otro
include + GetPayload	La relación en el JSON, y el tipo que la conoce
validations/ + validate	Un schema por operación, enganchado a la cadena de Express
errorHandler	Diez try/catch menos, y los status correctos
api.http	Ahora documenta los errores, no sólo el camino feliz



Lo que falta, falta a propósito

Hoy nos llevamos: un modelo con relaciones de verdad, una API que valida lo que entra y que devuelve el status correcto, y veinte líneas menos de código repetido.

Pendiente de hoy	**Lo resuelve**
Cualquiera borra cualquier libro. No hay usuarios	C18 · Auth con JWT + bcrypt
Tu front sigue leyendo libros.json	C19 · front ↔ back
El front no sabe que el JSON ahora trae autor anidado	C19 · ajustar los tipos del front
Nadie testea que el 400 siga siendo 400	C23 · Testing
Agregaciones (_count), select vs include, transacciones	Existen, están en la doc. No los necesitamos todavía

Actividades

Actividad Individual

1° instancia hasta el **30/08/2026**
*2° instancia: **13/09/2026** (Cierre Bloque 3)*

[AI-C17] Relaciones y validaciones en Librería

Referencia en el repo de ejemplo branch C17

En ds2026-actividades: copiar el sitio de la librería de la Clase 16 en **ejercicios/c17-relaciones-validaciones/**

2. schema.prisma con **al menos una 1:N y una N:M**: Autor 1:N Libro, Libro N:M Categoria

3. La migración commiteada, con nombre descriptivo

4. prisma/seed.ts reescrito: autores y categorías primero, libros con connect, **sin id a mano**

5. include en el service: el listado con su autor, el detalle con autor y categorías, tipados con GetPayload

6. validations/ con schemas de create y update para **libro y autor**, más el de params

7. middlewares/ con validate, validateParams y errorHandler, con el mapeo de P2002/P2025/P2003

8. **Cero try/catch** en los controllers

9. api.http corriendo entero, **con los casos de error**: 400, 404 y 409

10. Commiteá y avisá en #actividades

→ hilo **[AI-C17] Relaciones y validaciones en Librería**

Actividad Grupal

Hasta el **30/08/2026** (inclusive)

📌 **RECORDATORIO**
El **13/09/2026** es la **ENTREGA 3 (Backend)** del **Trabajo Práctico Grupal**

[AG-C17] Relaciones y validaciones en el TP

1. **Modelo:** traducir a schema.prisma las relaciones que dibujaron hoy en papel. Mínimo una 1:N. Si su dominio tiene una N:M, incluirla, y decidir si es implícita o explícita

2. **Migraciones** commiteadas. Si la base ya tenía datos, contar en el PR cómo lo resolvieron

3. **seed.ts** actualizado con las relaciones, con datos del dominio de su cliente

4. **onDelete** decidido a conciencia en cada relación, no por default. Justificarlo en una línea en el PR

5. **5 endpoints con validación** de Zod, incluyendo los params

6. **errorHandler** andando, con el mapeo de errores de Prisma

7. **api.http** con los casos de error de esas entidades

8. Repartir el trabajo, PRs con review cruzado (al menos uno cada integrante), y mover las tarjetas de Trello a "Done"

9. Aviso en #actividades
   → hilo **[AG-C17] Relaciones y validaciones en el TP**

⚠️

Antes de la próxima clase

⚠️ **INSTALAR ANTES DE C18**

El aula no tiene internet confiable.

Sin instalación = sin práctica.

```bash
# 1) Agregar las dependencias al proyecto
cd backend
npm install bcrypt jsonwebtoken
npm install -D @types/bcrypt @types/jsonwebtoken

# 2) Reconstruir la imagen ← este paso NO es opcional
cd ..
docker compose build api

# 3) Verificar que quedó adentro del contenedor
docker compose up -d
docker compose exec api npm ls bcrypt jsonwebtoken
```

✔ Dudas → canal #consultas de Discord

📅 C18: Seguridad y Autenticación (31/08/2026)